Teoria Programowania

Notatki z wykładów

„Informatyka jest ułomną nauką, bo niewiele umie”

Popełnione przez

Pontón Baitonio

Pompona Vanilla

Kraków

Anno Domini 2025

Spis treści

Licencja

Ten utwór jest dostępny na licencji Creative Commons Uznanie autorstwa na tych samych warunkach 4.0 Międzynarodowe.

Rozdział 1

Języki pierwotnie rekurencyjne

1.1 Rekursja prosta

Klasa funkcji pierwotnie rekurencyjnych $PRec$ to najmniejsza klasa zawierająca:

- $O^k(x_1, \ldots, x_n) = 0$, zero,
- $S(x) = x + 1$, następnik,
- $I_k^n(x_1, \ldots, x_n) = x_k$ dla $k \leq n$, rzutowania.

i zamknięta na schemat rekursji prostej

Twierdzenie 1.1.1 (Schemat rekursji (indukcji) prostej)**.** Niech dane będą zbiory A, Z oraz funkcje $g : A \to Z, h : Z \times \mathbb{N} \times A \to Z$.

Wtedy istnieje dokładnie jedna funkcja $f : \mathbb{N} \times A \to Z$, dla której zachodzi

1. $f(0, a) = g(a)$ – warunek początkowy (bazowy)
2. $f(n', a) = h(f(n, a), n, a)$ – rekursor

Przy pomocy tego schematu możemy zdefiniować podstawowe operacje dla $A = Z = \mathbb{N}$.

Dodawanie

$$\begin{cases} f(0, n) = g(n) = n \\ f(k + 1, n) = h(f(k, n), k, n) = f(k, n) + 1 \end{cases}$$

Dodawanie definiujemy jako $k + n = f(k, n)$

Mnożenie

$$\begin{cases} f(0, n) = g(n) = 0 \\ f(k + 1, n) = h(f(k, n), k, n) = f(k, n) + n \end{cases}$$

Mnożenie definiujemy jako $k \cdot n = f(k, n)$

Odejmowanie

$$xy = \begin{cases} x - y \text{ gdy } x \geq y \\ 0 \text{ gdy } x < y \end{cases}$$

Definiujemy funkcję pomocniczą $g(x) = x1$:

$$g(0) = 0 \qquad\qquad g(x+1) = x$$

Teraz możemy zdefiniować xy jako:

$$x0 = x \qquad\qquad x(y+1) = xy1$$

Ograniczony μ-operator

Definicja z ćwiczeń:
Funkcja $f : \mathbb{N}^k \to \mathbb{N}$ jest zdefiniowana za pomocą **ograniczonego μ-operatora** (**operacji minimum**) z funkcji $h : \mathbb{N}^k \to \mathbb{N}$ oraz $g : \mathbb{N}^{k+1} \to \mathbb{N}$, jeśli

$$f(\vec{n}) = \begin{cases} \min\{m \leq h(\vec{n}) : g(\vec{n}, m) = 0\}, & \text{jeśli } \exists_{m \leq h(\vec{n})}\ g(\vec{n}, m) = 0, \\ 0, & \text{wpp.} \end{cases}$$

Zapisujemy wtedy

$$f(n_1, \ldots, n_k) = \mu\, m_{\leq h(n_1, \ldots, n_k)}\ [\, g(n_1, \ldots, n_k, m) = 0\,].$$

Na ćwiczeniach (1.5) dowiedliśmy bardzo przydatne twierdzenie:

Twierdzenie 1.1.2. Jeśli f jest zdefiniowana z funkcji $g, h \in PRec$ za pomocą ograniczonego μ-operatora, to f jest pierwotnie rekurencyjna.

1.2 Predykaty pierwotnie rekurencyjne

Relacja $R \subset \mathbb{N}^{k+1}$ jest **relacją regularną** wtw:

$$\forall_{\vec{x} \in \mathbb{N}^k}\ \exists_{y \in \mathbb{N}}\ (\vec{x}, y) \in R$$

Predykat $R \subseteq \mathbb{N}^k$ jest pierwotnie rekurencyjny, gdy istnieje pierwotnie rekurencyjna funkcja charakterystyczna $f_R : \mathbb{N}^k \to \mathbb{N}$

$$f_R(\vec{x}) = \begin{cases} 1 \text{ gdy } \vec{x} \in R \\ 0 \text{ gdy } \vec{x} \notin R \end{cases}$$

Twierdzenie 1.2.1. Rodzina predykatów pierwotnie rekurencyjnych jest zamknięta na operacje logiczne.

Dowód. Definiujemy operacje logiczne za pomocą operacji z $PRec$.

- $a \wedge b :- a \cdot b$
- $\neg a :- 1a$
- $a \vee b :- \neg(\neg a \wedge \neg b)$

□

Twierdzenie 1.2.2. Rodzina predykatów pierwotnie rekurencyjnych jest zamknięta na kwantyfikatory ograniczone.

Dowód. Definiujemy kwantyfikatory ograniczone używając μ-rekursji:

$$\exists_{x \leq a} f(x) := sgn(f(0) + \mu x_{\leq a}(\neg f(x))$$

$$\forall_{x \leq a} f(x) := \overline{sgn}(\neg f(0) + \mu x_{\leq a}(f(x))$$

□

Rozdział 2

Funkcje rekurencyjne

2.1 Rekursja z historią

Kodowanie Gödla

Wprowadzamy oznaczenia:

$: \mathbb{N}^2 \to \mathbb{N}$ – funkcja Cantora

$, : \mathbb{N} \to \mathbb{N}$ – lewa i prawa wartość w odwrotnej funkcji Cantora, $((n), (n)) = n$

$(x, y) = x \pmod{y}$

Ciąg $x_1, \ldots, x_n$ kodujemy jako

$$\langle x_1, \ldots, x_n \rangle = p_0^n p_1^{x_1} \ldots p_n^{x_n},$$

gdzie $p_0, \ldots, p_n$ to kolejne liczby pierwsze.

Kodowanie, rozkodowywanie i poniższe operacje są pierwotnie rekurencyjne:

$\ln(x)$ – długość ciągu kodowanego przez x

$\mathrm{seg}(x)$ – sprawdzenie, czy x jest poprawnym kodem

xy – konkatenacja ciągów x i y

$x \sqsubseteq y$ – sprawdzenie, czy x jest podciągiem początkowym (?) y

$(((x)_i)_j)_k = (x)_{i,j,k}$ – dostęp do elementu $x[i][j][k]$

$\langle x \rangle_i$ – odkodowanie i-tego elementu ciągu

Funkcja β Gödla

Definiujemy funkcję $\beta : \mathbb{N}^2 \to \mathbb{N}$ jako

$$\beta(a, i) = a_i,$$

$$\beta(a, i) = \big((a), 1 + (i + 1) \cdot (a)\big),$$

gdzie a jest kodem ciągu $a_1, \ldots, a_n$ oraz $i < n$.

Kodowanie historii obliczeń

Dla $f^{k+1} : \mathbb{N}^{k+1} \to \mathbb{N}$ oraz $\vec{x} = (x_1, \ldots, x_k)$ definiujemy $\hat{f}^{k+1} : \mathbb{N}^{k+1} \to \mathbb{N}$ jako:

$$\hat{f}^{k+1}(\vec{x}, y) = \langle f(\vec{x}, 0), \ldots, f(\vec{x}, y)\rangle$$

Zapisujemy stan obliczeń funkcji f po n krokach:

$$s(\vec{x}, n) = \underbrace{\langle \vec{x}, n, f(\vec{x}, n)\rangle}_{k+1 \text{ wartości}}$$

Potrzebna jest jeszcze funkcja t, która, otrzymawszy stan po n krokach, oblicza stan po $n + 1$ krokach:

$$t(s(\vec{x}, n)) = s(\vec{x}, n + 1)$$

$$t(z) = \langle \underbrace{\langle z\rangle_0, \ldots, \langle z\rangle_k}_{\vec{x}}, \underbrace{\langle z\rangle_{k+1}}_{n}, \underbrace{h(\langle z\rangle_0, \ldots, \langle z\rangle_k, \langle z\rangle_{k+1}, \langle z\rangle_{k+2})}_{h(\vec{x},n)}\rangle$$

Twierdzenie 2.1.1 (O rekursji z historią, Thoralf Skolem)**.** Klasa $PRec$ jest zamknięta na rekursję z historią.

Dowód. Chcemy pokazać, że funkcja f zdefiniowana następująco:

$$\begin{cases} f(\vec{x}, 0) = g(\vec{x}) \\ f(\vec{x}, n + 1) = h(\vec{x}, n, \hat{f}(\vec{x}, n)) \end{cases}$$

jest pierwotnie rekurencyjna. Korzystamy z tego, że funkcja $\hat{f}$ może być wyrażona zgodnie ze schematem rekursji prostej:

$$\begin{cases} \hat{f}(\vec{x}, 0) = \langle g(\vec{x})\rangle \\ \hat{f}(\vec{x}, n + 1) = \hat{f}(\vec{x}, n)\langle \hat{f}(\vec{x}, n)\rangle, \end{cases}$$

więc jest w klasie $PRec$. Korzystając z tego, zapisujemy $f(\vec{x}, n) = \langle \hat{f}(\vec{x}, n)\rangle_{n+1}$, zatem $f \in PRec$. $\square$

2.2 Eliminacja rekursji prostej

Twierdzenie 2.2.1 (O eliminacji rekursji prostej)**.** Klasa funkcji rekurencyjnych (Rec) jest najmniejszą klasą zawierającą sumę, produkt, rzutowanie I_k^n, funkcje charakterystyczną równości σ i zamkniętą na złożenia i μ-rekursję

Dowód. Oznaczmy przez C klasę spełniającą założenia z twierdzenia. Z definicji $C \subseteq Rec$. Dowodzimy, że $Rec \subseteq C$.

Musimy poczynić pewne założenia, które później udowodnimy.

- Funkcja β-Gödla jest w C
- $\wedge$ jest w C
- ograniczony duży kwantyfikator $\forall_{y<z} R(\vec{x}, y)$ gdy $R \in C$ jest w C

Każdą funkcję z Rec możemy zdefiniować w C

1. Funkcji inicjujące
   - $O^n(\vec{x}) = \mu y[I_{n+1}^{n+1}(\vec{x}, y) = 0]$
   - $1^n(\vec{x}) = \mu y[(y \neq 0) = 0]$
   - $S(X) = I_1^1(x) + 1^1(x)$
2. Funkcje zdefiniowane za pomocą rekursji prostej Weźmy funkcję

$$f(\vec{x}, 0) = g(\vec{x})$$
$$f(\vec{x}, y+1) = h(\vec{x}, y, f(\vec{x}, y)). \text{ gdy } g, h \in C$$

   Jest to oczywiście funkcja zdefiniowana indukcją prostą. Pokażemy, że ta funkcja należy do C.

   Zbudujemy $t(\vec{x}, y) = z$, historię obliczenia f od 0 do y. Posiłkując się μ-rekursją przeglądamy liczby, aż do znalezienia takiej, która koduje historię:

$$z = \langle f(\vec{x}, 0), \ldots, f(\vec{x}, y) \rangle$$

$$t(\vec{x}, y) = \mu z[\beta(z, 0) = g(x) \wedge \forall_{i<y}\ \beta(z, i+1) = h(\vec{x}, i, \beta(z, i))]$$

$$f(\vec{x}, y) = \beta(t(\vec{x}, y), y)$$

Pozostało udowodnić założenia:

1. C jest zamknięta na operacje logiczne.
   - $\neg x :- (x = 0)$

- $x \wedge y :- x \cdot y$

2. C jest zamknięta na μ-operację na predykatach.

$$\mu y[R(\vec{x}, y)] = \mu y[f_R(\vec{x}, y) = 1] = \mu y[f_{\neg R}(\vec{x}, y) = 0]$$

3. C jest zamknięta na ograniczony duży kwantyfikator. Zdefiniujmy operację:

$$\mu^*_{y<z}[R(\vec{x}, y)] = \begin{cases} \mu y[R(\vec{x}, y)] \text{ gdy } \exists_y\ y < z \wedge R(\vec{x}, y) \\ z \quad \text{w.p.p} \end{cases}$$

μ^* jest w C bo możemy je zdefiniować jako

$$\mu^*_{y<z} R(\vec{x}, y) = \mu y[R(\vec{x}, y) \vee y = z]$$

Przy pomocy tej funkcji możemy zdefiniować ograniczone kwantyfikatory

$$\exists_{y<z}\ R(\vec{x}, y) :- [\mu^*_{y<z} R(\vec{x}, y)] \neq z$$

$$\forall_{y<z}\ R(\vec{x}, y) :- \neg\exists_{y<z}\ \neg R(\vec{x}, y)$$

4. C zawiera funkcję β Gödla.

$$J(x, y) = \frac{(x+y)^2 + 3xy}{2}$$

$$\mathcal{P}(z) = \mu y\ [\exists_{x<z+1}\ J(x, y) = z]$$

$$\mathcal{L}(z) = \mu x\ [\exists_{y<z+1}\ J(x, y) = z]$$

$$R_n(x, y) = \mu z\ [\exists_{p<y}\ xp + z = y]$$

$$\beta(a, i) = R_n\big(\mathcal{L}(a), 1 + (i+1) \cdot \mathcal{P}(a)\big)$$

□

2.3 Arytmetyzacja funkcji

Numerujemy wszystkie funkcje rekurencyjne:

- $\langle 0, n\rangle$ – numer zera O^n
- $\langle 1\rangle$ – numer następnika S
- $\langle 2, n, i\rangle$ – numer rzutowania $I^n_i, i \leq n$

- $\langle 3, b_1, \ldots, b_n, a\rangle$ – numer złożenia:

$$f(\vec{x}) = g(h_1(\vec{x}), \ldots, h_n(\vec{x})),$$

gdzie g ma numer a, h_i ma numer b_i.

- $\langle 4, a, b\rangle$ – numer funkcji f otrzymanej za pomocą rekursji prostej:

$$\begin{cases} f(\vec{x}, 0) = g(\vec{x}) \\ f(\vec{x}, y+1) = h(\vec{x}, y, f(x, y)) \end{cases}$$

a to numer funkcji g, a b to numer funkcji h.

- $\langle 5, a\rangle$ - numer funkcji otrzymywanej przez μ-rekursję:

$$f(\vec{x}) = \mu y[g(\vec{x}, y) = 0],$$

gdzie a to numer funkcji g.

Kodowanie nie jest suriektywne, ale możemy sprawdzić, czy kod jest poprawny, przeglądając kolejne funkcje. Dla $f \subseteq Rec$ przez e_f oznaczamy najmniejszy kod f.

2.4 Forma normalna rekursji

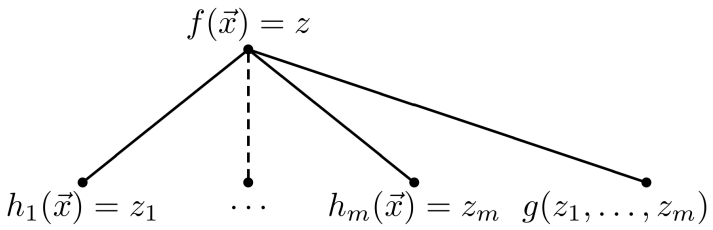
Twierdzenie 2.4.1 (O formie normalnej rekursji)**.** Istnieje funkcja $U : \mathbb{N} \to \mathbb{N}$, $U \in PRec$ oraz predykaty $\tau^n \in \mathbb{N}^{n+2}$ z $PRec$, takie że dla każdej funkcji $f : \mathbb{N}^n \to \mathbb{N} \in Rec$

- $\forall_{x_1,\ldots,x_n} \exists_y \ \tau^n(e_f, x_1, \ldots, x_n, y)$
- $f(x_1, \ldots, x_n) = U\big(\mu y[\tau^n(e_f, x_1, \ldots, x_n, y)]\big)$

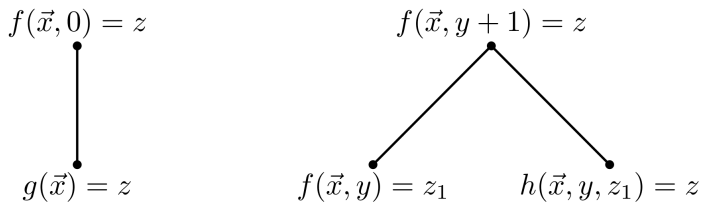
Liczba y to numer obliczenia funkcji o numerze e_f na wejściu $\vec{x}$. Wtedy $\mu y[\tau^n(e_f, \vec{x}, y)]$ znajduje numer tej historii obliczeń, a U odzyskuje z niej wynik.

Dowód. Definiujemy drzewa obliczeń funkcji f:

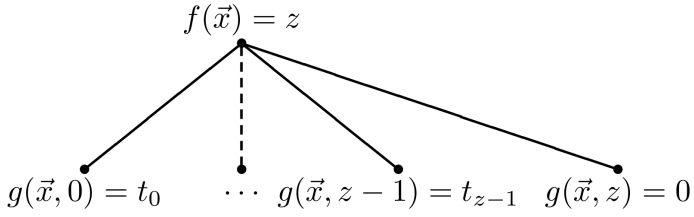
- Liście: $f(x) = 0$, $f(x) = x + 1$, $f(\vec{x}) = x_k$ dla funkcji inicjujących
- Drzewo dla złożenia $f(\vec{x}) = g(h_1(\vec{x}), \ldots h_k(\vec{x}))$:



- Drzewo obliczeń dla rekursji prostej



- Drzewo dla μ-rekursji

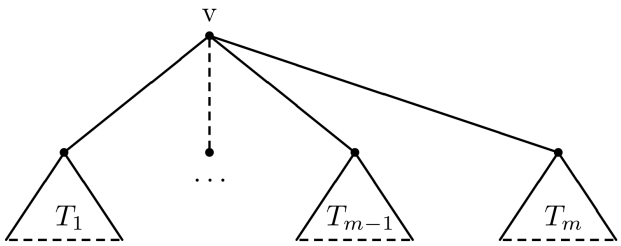


Pozostało tylko ponumerować drzewa. Wierzchołek v:

$$v = [f(x_1, \ldots, x_n) = z]$$

numerujemy jako:

$$\hat{v} = \langle e_f, \langle x_1, \ldots, x_n \rangle, z \rangle$$



Drzewa kodujemy jako $\hat{T} = \langle \hat{v}, \hat{T}_1, \ldots, \hat{T}_k \rangle)$

Konstruujemy predykat $\mathcal{T}(y) \in Prec$ rozumiany jako *y jest kodem drzewa obliczeń.*

$A(y)$ – wstępne warunki

$$A(y) = \underbrace{(y)}_{\text{kod drzewa}} \wedge \underbrace{((y)_1) \wedge \big(((y)_1) = 3\big)}_{\text{kod } v} \wedge \underbrace{((y)_{1,1})}_{\text{kod } f} \wedge \underbrace{((y)_{1,2})}_{\text{argumenty}}$$

$B(y)$ – warunki poprawnego kodowania funkcji inicjujących
$C(y)$ – warunki poprawnego kodowania złożenia
$D(y)$ – warunki poprawnego kodowania rekursji prostej
$E(y)$ – warunki poprawnego kodowania μ-operatora

Korzystając z powyższych definiujemy predykat $\mathcal{T}$ jako:

$$
\begin{aligned}
&\mathcal{T}(y) \iff \\
&A(y) \wedge && \text{(poprawność kodu)} \\
&\big[B(y) \vee C(y) \vee D(y) \vee E(y)\big] \wedge && \text{(dopasowanie którejś z dopuszczalnych funkcji)} \\
&\big[(y) > 1 \to \forall_{2 \leq i \leq (y)}\ T(y(i))\big] && \text{(poprawność poddrzew)}
\end{aligned}
$$

Odwołanie do $\mathcal{T}(y(i))$ nie jest problemem, bo możemy korzystać z rekursji z historią, a $y(i) < y$.

W ten sposób otrzymujemy $\tau^n, U \in \prec$:

$$
\begin{aligned}
&\tau^n(e_f, \vec{x}, y) = \\
&\mathcal{T}(y) \wedge \\
&\big[(y)_{1,1} = e_f\big] \wedge && \text{(poprawny numer funkcji)} \\
&\big[(y)_{1,2} = \langle \vec{x} \rangle\big] && \text{(zgodność argumentów)} \\
\\
&U(y) = (y)_{1,3}
\end{aligned}
$$

$\square$

Rozdział 3

Rachunek lambda

Po podstawowe definicje, czym są termy lambda, odsyłam do notatek z ćwiczeń.

Podstawienie $M[x := n]$ oznacza term powstały z M poprzez zastąpienie wszystkich wolnych wystąpień x przez N.

α-konwersja

Operacja zmiany nazw zmiennych związanych

$$\begin{aligned} &M =_\alpha M \\ &\lambda x.M =_\alpha \lambda y.M[x := y] \quad \text{dla } y \notin FV(M) \end{aligned}$$

β-redukcja

Operacja przekształcenia termu w postaci **redeks** $((\lambda x.M)N)$ poprzez zastąpienie wolnych wystąpień zmiennej argumentem

$$(\lambda x.M)N \to_\beta M[x := N]$$

Warunek: żadna zmienna z $FV(N)$ nie zostanie związana po podstawieniu do M.

Przykład, jak nie można:

$$(\lambda xy.xy)y \overset{\times}{\to}_\beta \lambda y.yy$$

Należy wcześniej zastosować α-konwersję.

$$(\lambda xy.xy)y =_\alpha (\lambda xz.xz)y \to_\beta \lambda z.yz$$

Jeśli $M \to_\beta N$, to

- $\lambda x.M \to_\beta \lambda x.N$

- $MK \to_\beta NK$
- $KM \to_\beta KN$

Definiujemy:
$\twoheadrightarrow_\beta$ domknięcie przechodnie i zwrotne relacji $\to_\beta \cup =_\alpha$,
$=_\beta$ domknięcie symetryczne relacji $\twoheadrightarrow_\beta$.

Twierdzenie 3.0.1 (Church-Rosser, 1936)**.** Dla dowolnych termów M, N, K, które spełniają $M \twoheadrightarrow_\beta N$ i $M \twoheadrightarrow_\beta K$ istnieje term L taki, że $N \twoheadrightarrow_\beta L$ i $K \twoheadrightarrow_\beta L$.

$$\begin{tikzcd}
 & N \arrow[dr, two heads, "\beta"] & \\
M \arrow[ur, two heads, "\beta"] \arrow[dr, two heads, "\beta"'] & & L \\
 & K \arrow[ur, two heads, "\beta"'] &
\end{tikzcd}$$

Twierdzenie 3.0.2 (O zygzaku, Church-Rosser)**.** Jeśli $M =_\beta N$, to istnieje zygzak.

Szkic. Wynika z poprzedniego twierdzenia.

$$\begin{tikzcd}
M \arrow[dr, two heads, "\beta"'] & & K_2 \arrow[dl, two heads, "\beta"'] \arrow[dr, two heads, "\beta"] & & N \\
 & K_1 & & K_3 \arrow[ur, dashed, two heads] &
\end{tikzcd}$$

□

Twierdzenie 3.0.3 (Church-Rosser)**.** Dla dowolnych termów $M =_\beta N$ istnieje L, dla którego $M \twoheadrightarrow_\beta L \twoheadleftarrow_\beta N$.

Szkic. Wynika z poprzedniego twierdzenia.

$$\begin{tikzcd}
 & & & & & \\
M \arrow[ur, two heads] \arrow[dr, two heads] & & \arrow[ul, two heads] \arrow[dr, two heads] & & \arrow[ul, two heads] \arrow[dr, two heads] & & N \arrow[ul, two heads] \\
 & \arrow[ul, two heads] \arrow[dr, two heads] & & \arrow[ul, two heads] \arrow[dr, two heads] & & \arrow[ul, two heads] & \\
 & & \arrow[ul, two heads] \arrow[dr, two heads] & & \arrow[ul, two heads] & & \\
 & & & L \arrow[ur, two heads] & & &
\end{tikzcd}$$

□

Term M jest w formie normalnej, jeśli nie posiada redeksów.

Twierdzenie 3.0.4 (O formie normalnej, Church-Rosser)**.** Dla dowolnych termów M, N, K, które spełniają $M \twoheadrightarrow_\beta N$ i $M \twoheadrightarrow_\beta K$, jeśli N i K są normalne, to $N =_\beta K$.

Szkic. Na mocy pierwszego twierdzenia Churcha-Rossera istnieje L takie, że $N \twoheadrightarrow_\beta L$ i $K \twoheadrightarrow_\beta L$. Skoro N i K są normalne, to nie zawierają redeksów, więc dalsze β-redukcje są niemożliwe, czyli $N \twoheadrightarrow_\alpha L$ i $K \twoheadrightarrow_\alpha L$, a zatem $N =_\beta K$.

$$\begin{tikzcd}
 & N \arrow[dr, two heads, "\alpha"] & \\
M \arrow[ur, two heads, "\beta"] \arrow[dr, two heads, "\alpha"'] & & L \\
 & K \arrow[ur, two heads, "\alpha"'] &
\end{tikzcd}$$

□

Ważne termy

- $K = \lambda xy.x = \pi_i$
- $K^* = \lambda xy.y = \pi_2$
- Para uporządkowana:

$$\lambda yzx.xyz = U$$

$$[p, q] = Upq = \lambda x.xpq$$

$$[p, q]\pi_1 = p \qquad [p, q]\pi_2 = q$$

- $S = \lambda xyz, xz(yz)$
- $\omega = \lambda x.xx$
- $\Omega = \omega\omega = (\lambda y.yy)(\lambda x.xx) \to_\beta (\lambda x.xx)(\lambda x.xx) \to_\beta \ldots$ najprostszy term bez postaci normalnej
- Operator punktu stałego Y to taki term, że dla każdego termu M

$$YM =_\beta M(YM)$$

najkrótszym operatorem punktu stałego (potwierdzone bruteforcem) jest:

$$Y = \lambda y(\lambda x.y(xx))(\lambda x.y(xx))$$

3.1 Numerale Churcha

Numerale Churcha reprezentują liczby naturalne w rachunku lambda, można by je zdefiniować inaczej, ale tak jest wygodnie.

Definiujemy liczby naturalne jako

$$0 \equiv \lambda sz.z$$

$$1 \equiv \lambda sz.sz$$

$$n \equiv \lambda sz.\underbrace{s(s\ldots(s}_{\text{n razy s}}\,z))$$

Interpretacja jest taka, że bierzemy termy z reprezentujący zero oraz s reprezentujący następnik i aplikujemy następnik n razy do zera.
Numerale Churcha są w postaci normalnej więc $i \neq j \Rightarrow i \neq_\beta j$.

3.2 λ-definiowalność

Funkcja $f : \mathbb{N}^k \to \mathbb{N}$ jest λ-definiowalna przez term domknięty F jeśli dla dowolnych liczb $n_1, \ldots, n_k$

$$Fn_1n_2\ldots n_k =_\beta f(n_1, \ldots, n_k)$$

Twierdzenie 3.2.1 (Church)**.** Funkcje rekurencyjne są λ-definiowalne.

Dowód. Dowodzimy indukcją strukturalną po funkcjach rekurencyjnych:

- Funkcje inicjujące
  - $\lambda x_1\ldots x_k.0$ – zero
  - $\lambda nsz.s(nsz)$ – następnik
  - $\lambda x_1\ldots x_k.x_i,\ i \leq k$ – rzutowania
- Domknięcie na złożenia

$$g \equiv \lambda x_1\ldots x_k.G(F_1x_1\ldots x_k)\ldots(F_nx_1\ldots x_k)$$

- Domknięcie na rekursję prostą

$$\begin{aligned} M = \lambda Fnx_1\ldots x_k.&\text{IF (iszero } n\text{) THEN } Gx_1\ldots x_k \\ &\text{ELSE } H\text{ (pred } n\text{) } x_1\ldots x_k\,(F(\text{pred } n)x_1\ldots x_k), \\ f \equiv YM& \end{aligned}$$

Dowód indukcyjny:
$YM\,0\,x_1 \ldots x_k =_\beta M(YM)\,0\,x_1 \ldots x_k =_\beta Gx_1 \ldots x_k =_\beta g(x_1, \ldots, x_k) =_\beta f(0, x_1, \ldots, x_k)$

$$\begin{aligned} YM\,(n+1)\,x_1 \ldots x_k &=_\beta M(YM)\,(n+1)\,x_1 \ldots x_k \\ &=_\beta Hnx_1 \ldots x_k((YM)nx_1 \ldots x_k) \\ &=_\beta Hnx_1 \ldots x_k(f(n, x_1, \ldots, x_k)) \text{ (z zał. ind.)} \\ &=_\beta h(n, x_1, \ldots, x_k, f(n, x_1, \ldots, x_k)) \\ &=_\beta f(n, x_1, \ldots, x_k) \end{aligned}$$

- Domknięcie na μ-rekursję

$$M = \lambda Gy.\text{IF } (Gy) \text{ THEN } y \text{ ELSE } G(y+1)$$

$$f \equiv YM$$

□

Twierdzenie 3.2.2 (Church, 1936)**.** Każda funkcja rekurencyjna ma λ-reprezentację.

Twierdzenie 3.2.3. Każda funkcja λ-reprezentowalna jest rekurencyjna.

3.3 Arytmetyzacja rachunku lambda

Enumerujemy symbole:

znak x_i	$\cdot$	(	)	λ	x_1	x_2	...
kod α_i	2	3	5	7	11	13	...

Kodujemy skończone typy:
Ciąg $x_1 \ldots x_n$ kodujemy jako $p_1^{\alpha_1} \ldots p_n^{\alpha_n}$.

Definiujemy $q, r \in PRec$ dla termów S, T w postaci normalnej kodowanych przez n, k:
$q(n, k)$ – numer redukcji $T \to_{\alpha\beta} S$
$r(n, k)$ – numer równości $T =_{\alpha\beta} S$
Definicje są poprawne i q, r rzeczywiście są rozstrzygalne, ponieważ możemy ponumerować wszystkie postaci normalne.

Konstruujemy rodzinę predykatów $\tau^n \subseteq \mathbb{N}^{n+2}$:

$$\tau^n(e, x_1, \ldots, x_n, y),$$

oznacza, że y jest numerem redukcji termu o numerze e do termu $x_1, \ldots, x_n$.

Konstruujemy funkcję $U : \mathbb{N} \to \mathbb{N}$:

$$U(y) = n, \text{ gdy } y \text{ to numer redukcji do } n$$
$$f(x_1, \ldots, x_n) = U(n, y, \tau^n(e, x_1, \ldots, x_n), y)$$

Rozdział 4

Rozstrzygalność

4.1 Rozstrzygalność problemów

Rozstrzygalność rozumiemy następująco:

$$\begin{array}{l} Rec - \text{rozstrzygalne} \\ \left.\begin{array}{l} RE - \text{częściowo rozstrzygalne} \\ \overline{RE} - \text{nierozstrzygalne} \end{array}\right\} \text{nierozstrzygalne} \end{array}$$

4.1.1 Język diagonalny

Definiujemy język diagonalny

$$L_D = \{w_i \in \Sigma_i^* : w_i \notin L(M_i)\}$$

Twierdzenie 4.1.1. $L_D \notin Rec$

Dowód. Nie wprost, załóżmy, że $L_D \in REC$, istnieje więc maszyna M_i, która go rozstrzyga.

- Jeśli $w_i \in L(M_i)$, to $w_i \notin L_D = L(M_i)$
- Jeśli $w_i \notin L(M_i)$, to $w_i \notin L_D = L(M_i)$

□

4.1.2 Język uniwersalny

$$L_u = \{\langle M \rangle, w : w \in \Sigma^*, w \in L(M)\}$$

Twierdzenie 4.1.2. $L_U \in RE$

Dowód. Konstruujemy trzytaśmową maszynę rozstrzygającą L_U:

1. Wydziel kod maszyny $\langle M \rangle$ i słowo $w \in \Sigma^*$.
2. Symuluj M na w przez i kroków
   (a) Jeżeli M akceptuje w po i krokach to zatrzymaj się i zwróć tak
   (b) W przeciwnym przypadku przypisz $i = i + 1$ i wróć do 2(a).

Skonstruowana w dowodzie maszyna to uniwersalna maszyna Turinga M_U, $L_U = L(M_U)$. Taka maszyna może i umie zasymulować każdą inną maszynę Turinga. □

Twierdzenie 4.1.3. $L_U \notin Rec$

To wynika z Twierdzenia Rice'a, które już niedługo :)

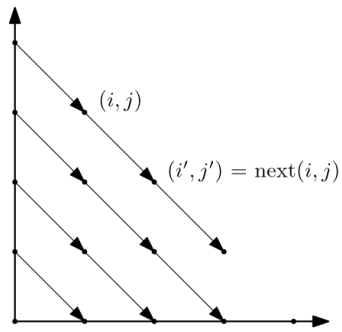
4.1.3 (Nie)pustość języków

$$L_e = \{\langle M \rangle : L(M) = \varnothing\}$$
$$L_{ne} = \{\langle M \rangle : L(M) \neq \varnothing\}$$

Twierdzenie 4.1.4. $L_{ne} \in RE$

Dowód. Konstruujemy maszynę rozpoznającą L_{ne}:

1. Wczytaj $\langle M \rangle$ i sprawdź, czy jest poprawnym kodem.
2. Symuluj j kroków maszyny M na i-tym słowie wg porządku (i, j), zaczynając od $(0, 0)$.
3. Jeśli M akceptuje słowo, to zaakceptuj $\langle M \rangle$.



□

Twierdzenie 4.1.5. $L_e \notin RE$

Dowód. Załóżmy dla dowodu nie wprost, że $L_e \in Rec$. Konstruujemy maszynę N z własnością STOP, która akceptuje wszystko, jeśli $w \in L(M)$ i nic wpp.

Maszyna N rozpoznaje $L_U \in RE$, co prowadzi do sprzeczności.

Zatem $L_e \notin Rec$, a ponieważ $L_{ne} = \Sigma^* \setminus L_e \in RE$, to L_e jest całkiem nierozstrzygalny. □

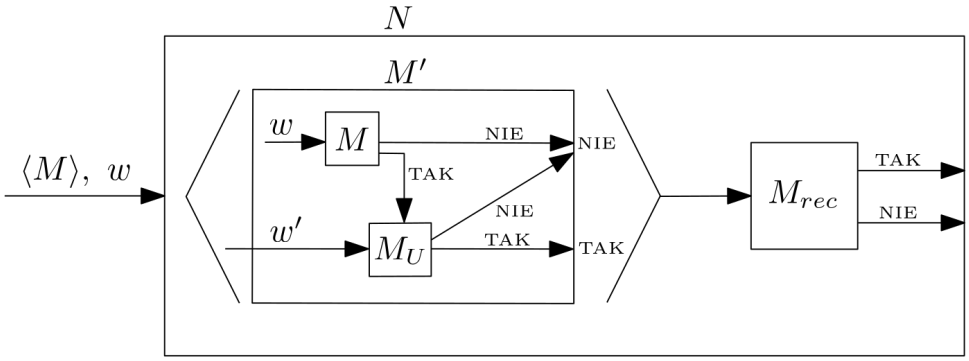
4.1.4 Rekurencyjność języków

$$L_{rec} = \{\langle M \rangle : L(M) \in Rec\}$$
$$L_{nrec} = \{\langle M \rangle : L(M) \notin Rec\}$$

Przykład nierozstrzygalnych języków, których suma $L_{rec} \cup L_{nrec} = \{\langle M \rangle\}$ jest rozstrzygalna.

Twierdzenie 4.1.6. $L_{rec} \notin RE$

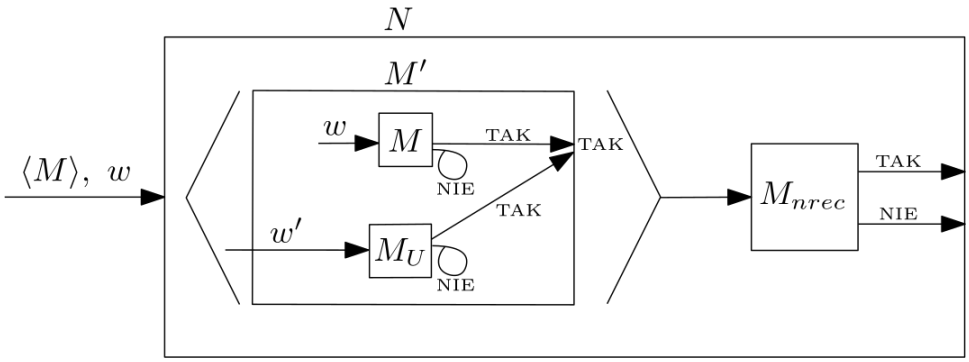
Dowód. Załóżmy dla dowodu nie wprost, że $L_{rec} \in RE$. Konstruujemy maszynę N:



Jeśli $w \in L(M)$, to M' akceptuje język nierekurencyjny (dlaczego?), a jeśli $w \notin L(M)$, to $L(M') = \varnothing \in R$. Zatem N rozpoznaje język $\overline{L_U}$, co prowadzi do sprzeczności. □

Twierdzenie 4.1.7. $L_{nrec} \notin RE$

Dowód. Załóżmy dla dowodu nie wprost, że $L_{nrec} \in RE$. Konstruujemy maszynę N:



Symulujemy na zmianę po jednym kroku M i M_U. Jeśli M akceptuje w, to maszyna M' akceptuje wszystko, niezależnie od M_U, a wtedy $L(M') = \Sigma^*$, czyli nie jest nierekurencyjny, bo maszyna, która akceptuje wszystko bez dodatkowych obliczeń ma własność STOP.
Jeśli $w \notin L(M)$, to wynik M' zależy jedynie od M_U, więc jest językiem nierekurencyjnym. Zatem N rozpoznaje język $\overline{L_U}$, co prowadzi do sprzeczności. □

4.2 Nierozstrzygalność

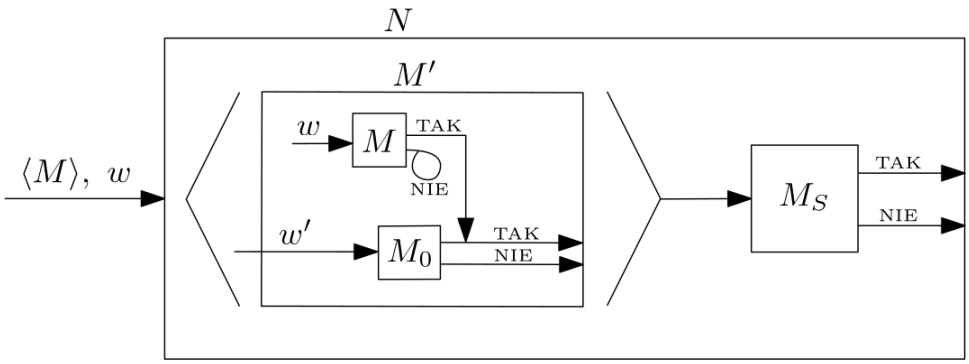
4.2.1 Pierwsze twierdzenie Rice'a

Dla $S \subseteq RE$ definiujemy $L_S = \{\langle M \rangle : L(M) \in S\}$.
Własność S jest trywialna, jeśli $S = \varnothing$ lub $S = RE$. Takie własności są rozstrzygalne – jeśli $S = \varnothing$, to $L_S = \varnothing$ i jest rozpoznawana przez maszynę z własnością STOP, która odrzuca wszystkie słowa, a jeśli $S = RE$, to L_S zawiera wszystkie poprawne kody maszyn, więc do rozpoznania L_S wystarczy parser.

Twierdzenie 4.2.1. (Rice) Każda nietrywialna własność języków rekurencyjnie przeliczalnych jest nierozstrzygalna, czyli poza Rec.

Dowód. Niech S będzie nietrywialną własnością. Załóżmy, że $\varnothing \notin S$. Możemy tak zrobić, ponieważ dopełnienie nietrywialnej własności również jest nietrywialne, więc jeśli $\varnothing \in S$ prowadzimy analogiczne rozumowanie dla dopełnienia S.

Niech $L_0 = L(M_0) \in S$. Dla dowodu nie wprost załóżmy, że S jest rozstrzygalna. Konstruujemy maszynę N z własnością stopu:



Jeśli M akceptuje w, to $L(M') = L_0$, więc M_S akceptuje. W przeciwnym przypadku $L(M') = \varnothing$, więc M_S nie akceptuje z założenia, że $\varnothing \notin S$. Maszyna N rozpoznaje język L_U, który nie jest rozstrzygalny – sprzeczność. □

4.2.2 Drugie twierdzenie Rice'a

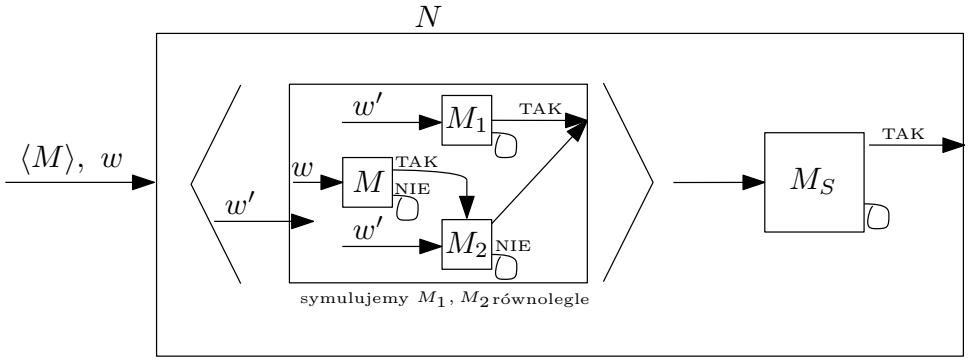
Monotoniczność

$$(L \in S \wedge L \subset L' \in RE) \iff L' \in S$$

Lemat 4.2.1. Jeśli S nie jest monotoniczna, to $L_S \notin RE$.

Dowód. Nie wprost załóżmy, że S nie jest monotoniczna oraz $L_S \in RE$. Istnieje maszyna M_S, która rozpoznaje L_S.

Weźmy $L_1 \subset L_2$, takie że $L_1 \in S, L_2 \notin S$ oraz są rozpoznawane przez maszyny M_1, M_2.



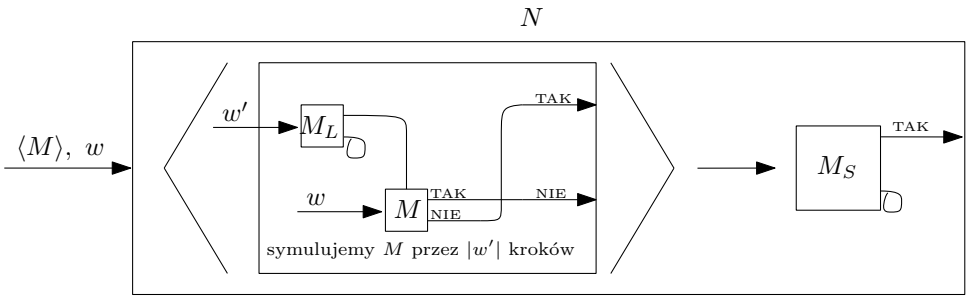
Symulujemy M_1, M_2 równolegle. Jeśli M akceptuje w, to $L(M') = L_2$. Jeśli nie akceptuje, to $L(M') = L_1$, więc M_s akceptuje. Zatem N rozpoznaje $\overline{L_U}$ – sprzeczność. □

Własność skończonego podjęzyka

$$\forall_{L \in S}\ \exists_{L' \subseteq L}\ L' \in S \wedge L' \text{ skończony}$$

Lemat 4.2.2. Jeśli S nie ma własności skończonego podjęzyka, to $L(S) \notin RE$.

Dowód. Nie wprost załóżmy, że $L \in S$ i nie ma własności skończonego podjęzyka.



Jeśli M akceptuje w w k krokach, to $L(M') = L \cap \{w\ : |w| < k\}$, czyli skończony podjęzyk L. Ponieważ S nie ma własności skończonego podjęzyka, to N się zapętli.

Jeśli M nie akceptuje w, to $L(M') = L \in S$ i N akceptuje. Czyli N rozpoznaje $\overline{L_U}$ – sprzeczność. □

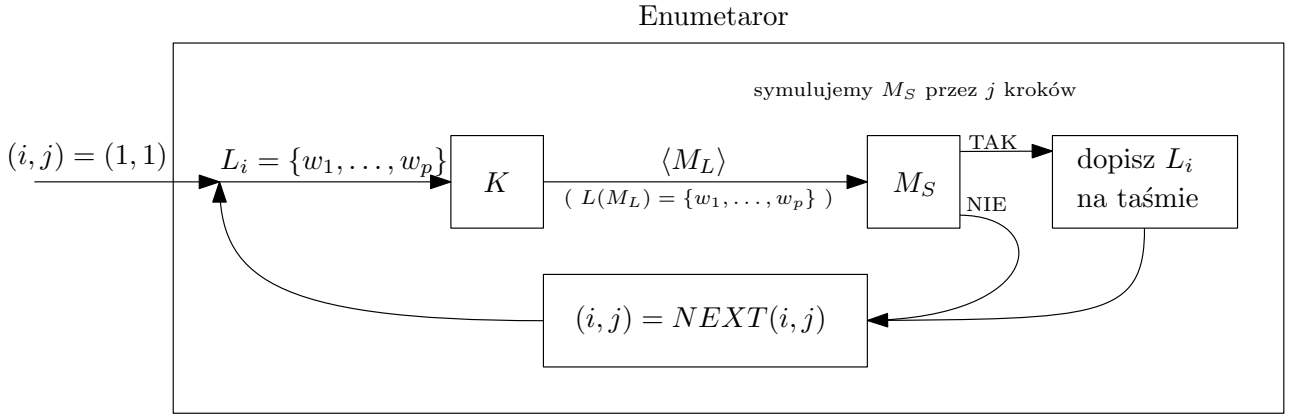
Uniwersalność

Klasa skończonych języków S jest uniwersalna, jeśli istnieje maszyna Turinga (numerator) bez wejścia, wypisująca suriektywnie języki skończone z S.

$$code_1\#code_2\#code_3\#\ldots \qquad code_i = w_1\$w_2\$\ldots\$w_p$$

Lemat 4.2.3. (O numeratorze) Jeśli $S \subset RE$, to istnieje numerator dla skończonych języków z S.

Dowód. Przechodzimy po $(i, j) \in \mathbb{N}^2$ przekątniowo. Maszyna K, dostawszy kod skończonego języka L, zwraca maszynę M_L, która go rozpoznaje. □



Twierdzenie 4.2.2. (Rice) Język $L_S = \{\langle M \rangle : L(M) \in S\}$ jest w klasie RE wtw S jest monotoniczna, uniwersalna i ma własność skończonego podjęzyka.

Dowód. Jeśli $L_S \in RE$, to warunki są spełnione, co wynika z lematów.

Dowodzimy, że jeśli warunki są spełnione, to $L_S \in RE$. Konstruujemy rozpoznawacza M_S.

Na wejściu jest $\langle M \rangle$ z numeratorem M_{NUM}. Przechodzimy przekątnymi po (i, j).

1. Wykonujemy i kroków M_{NUM}. Oznaczamy ostatni kompletny wypisany język na taśmie jako $L = \{w_1, \ldots, w_p\}$.
2. Każde ze słów $w_1, \ldots, w_p$ sprawdzamy przez j kroków na maszynie M.
3. Jeśli M akceptuje wszystkie, to zatrzymujemy maszynę i akceptujemy M.
4. Jeśli nie, to wracamy do początku z $(i, j) = \text{NEXT}(i, j)$.

Jeżeli $L(M) \in S$, to M_S się zatrzymuje i akceptuje M, bo akceptuje jakiś skończony podjęzyk $L(M)$. Jeśli M_S się zatrzymuje i akceptuje, to $L(M) \in S$ z monotoniczności. □

Każda z własności jest niezależna – dwie pozostałe jej nie implikują.

Rozdział 5

Maszyny z wyrocznią

5.1 Hierarchia nierozstrzygalności

5.1.1 Maszyna z wyrocznią

Dla języka A maszyna T^A oznacza maszynę z wyrocznią dla języka A. Wyrocznia ma własność STOP a zapytania są wykonywane w czasie $\mathcal{O}(1)$.

Język L jest w klasie RE (Rec) z wyrocznią dla A, gdy $L = L(T^A)$ dla maszyny T^A (z własnością STOP).

Dla języków $L_1, L_2 \subseteq \Sigma^*$ definiujemy relację równoważności:

$$L_1 \approx L_2 \text{ wtw } L_1 \text{ jest rekurencyjny z wyrocznią } L_2 \text{ i vice versa}$$

5.1.2 Hierarchia

Coraz głębsze warstwy nierozstrzygalności:

$$\begin{aligned} S_1 &= \{\langle M \rangle : L(M) = \varnothing\} \approx L_e \\ S_2 &= \left\{\langle M^{S_1} \rangle : L(M^{S_1}) = \varnothing\right\} \\ &\approx \{\langle M \rangle : L(M) = \Sigma^*\} \\ &\dots \end{aligned}$$

Kolejne warstwy korzystają z wyroczni dla poprzednich.

Dla każdego $i \geq 1$, S_{i+1} nie jest RE ze względu na S_i.

Twierdzenie 5.1.1. $L_u \approx L_e$

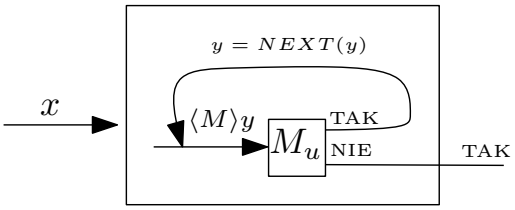
Dowód. Algorytm dla L_U przy wyroczni L_e
Konstruujemy maszynę M', która akceptuje Σ^*, jeśli $w \in L(M)$ i nic, wpp. Testujemy M', zapytując wyrocznię dla L_e. W ten sposób rekurencyjnie rozpoznajemy L_U.

Algorytm dla L_e przy wyroczni L_U
Konstruujemy maszynę M' taką, że albo $L(M) = L(M') = \varnothing$, albo $L(M) \neq \varnothing$ i $L(M') = \Sigma^*$. Iterując po parach (i, j), symulujemy j kroków M na i-tym słowie. Jeśli M zaakceptuje jakieś słowo, M' akceptuje słowo w' na wejściu, wpp M' się pętli.

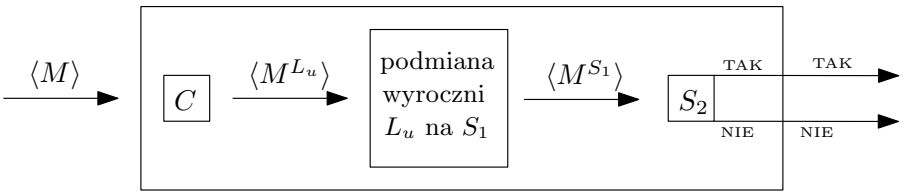
Maszyna M_U rozpozna $(\langle M' \rangle, w')$ wtw, gdy $L(M) \neq \varnothing$. □

Twierdzenie 5.1.2. $S_2 = \left\{\langle M^{S_1} \rangle : L(M^{S_1}) = \varnothing\right\} \approx \{\langle M \rangle : L(M) = \Sigma^*\}$

Dowód rekurencyjności $\geq$.
Konstruujemy pisarza C, który zwraca kod maszyny M^{L_u}, korzystającej z wyroczni L_u.



Następnie konstruujemy maszynę z wyrocznią dla S_1, która sprawdza, czy język maszyny jest pełny. Korzystamy z tego, że $L_u \approx S_1$, więc możemy podmienić wyrocznie L_u na wyrocznię S_1.



I Jeśli $L(M) = \Sigma^*$, to $L(M^{S_1}) = \varnothing$, więc maszyna akceptuje.

II Jeśli $L(M) \neq \Sigma^*$, to $L(M^{S_1}) = \Sigma^*$, więc maszyna nie akceptuje.

□

Rozdział 6

Przebudowa maszyn

6.0.1 Twierdzenie $S_{m,n}$

Maszyna M oblicza $f^{(k)}(n_1, \ldots, n_k) = p$, gdy na słowie $0^{n_1}1\,0^{n_2}1\ldots$ zatrzymuje się z wypisanym $f^{(k)}(n_1, \ldots, n_k)$ na taśmie wyjściowej.

$f_M^{(k)}$ to funkcja liczona przez maszynę M o k argumentach.
$f_m^{(k)}$ to funkcja liczona przez m-tą w numeracji maszynę o k argumentach.

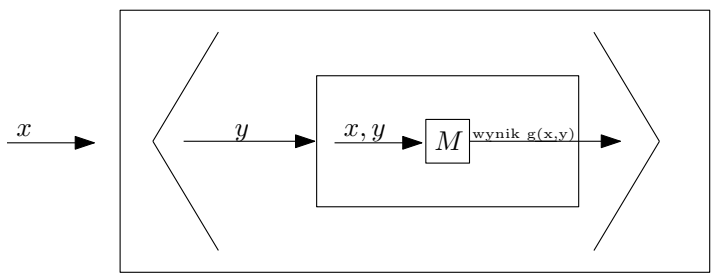
Twierdzenie 6.0.1 ($S_{m,n}$)**.** Dla częściowej funkcji rekurencyjnej $g : \mathbb{N}^2 \to \mathbb{N}$ istnieje totalna i rekurencyjna funkcja $\sigma : \mathbb{N} \to \mathbb{N}$, która spełnia

$$g(x, y) = f_{\sigma(x)}^{(1)}(y)$$

Dowód. Niech M oznacza maszynę, która oblicza g. Konstruujemy maszynę dla $\sigma(x)$:

1. Wczytaj x.
2. Skonstruuj maszynę, która przyjmuje y i symuluje x, y na M.
3. Oblicz indeks skonstruowanej maszyny, korzystając z enumeracji częściowych funkcji rekurencyjnych.

Maszyna dla $\sigma(x)$



□

Twierdzenie 6.0.2 (O punkcie stałym rekursji)**.** Dla totalnej funkcji rekurencyjnej $\sigma : \mathbb{N} \to \mathbb{N}$ istnieje konstruktywnie $x_0 \in \mathbb{N}$ takie, że

$$f_{x_0}(x) = f_{\sigma(x_0)}(x),$$

czyli obie zatrzymają się i wyniki będą równe.

Oznacza to, że dla każdej transformacji σ istnieje maszyna, która nie podlega transformacji i możemy ją skonstruować.

Dowód. Definiujemy G, konstruktora maszyn:

1. Wczytaj i.
2. Skonstruuj maszynę $M^{(i)}$ (opis poniżej).
3. Oblicz indeks $M^{(i)}$.

Maszyna $M^{(i)}$:

1. Wczytaj x.
2. Skonstruuj i-tą maszynę M_i.
3. Oblicz $k = f_i(i)$, symulując M_i na wejściu i (może się zapętlić).
4. Skonstruuj k-tą maszynę M_k.
5. Oblicz i wypisz $f_k(x)$ (może się zapętlić).

Jeśli maszyna G opisuje pewną funkcję totalną $g : \mathbb{N} \to \mathbb{N}$, to

$$f_{g(i)}(x) = f_{f_i(i)}(x)$$

Ustalając j równego indeksowi funkcji $\sigma \circ g$ oraz $x_0 = g(j)$, otrzymujemy:

$$f_{x_0}(x) = f_{g(j)}(x) = f_{\sigma(g(j))}(x) = f_{\sigma(x_0)}(x)$$

□